Homework 6

For this homework you will create a gitHub repo (or use the one from homework 5) and save a python notebook (`.ipynb` file) using Google Colab there. We'll also create a `.py` file for part II. This should be saved in your repo as well. You'll then submit a link to your gitHub repo. Be sure to:

- change the sharing settings on your notebook. This can be done by clicking on the 'share' icon in the top right. Please make sure to change the sharing settings so that anyone with the link can see the notebook.
- **include markdown text describing what you are doing, even when not explicitly asked for!**
- Add comments to your `.py` file explaining what you are doing.

The purpose of this homework is to get a little more practice with SQL and gain some practice with creating classes.

Part I - More Practice Querying a Database (16 pts)

There is a database file on the assignment link called `lahman_1871-2022.sqlite` that is an sqlite database downloaded from here. This database has information on Major League Baseball.

1. Connect to the database and then look at all of the tables in the database (use `read_sql()` from `pandas` to have this returned as a data frame). (2 pts)
2. Using SQL, construct a table of hall of fame pitchers (any hall of famer that pitched) that gives the `playerID` and their total (sum) for `GS`, `G`, `W`, `L`, `IPOuts`, `CG`, `SHO`, and `SV` columns. The summing can be done in `pandas` or in the SQL call. (6 pts)
3. For all of the hall of fame pitchers, use SQL to create a table of their batting statistics. Namely, the `playerID` and their total (sum) for `AB`, `R`, `H`, `HR`, `RBI`, `BB`, and `SO`. The summing can be done in `pandas` or in the SQL call. (4 pts)
4. Using `pandas` join the previous two tables together by pitcher. (If you want, try to do all of this via SQL! Not required though, feel free to use `pd.merge()` if you'd like) (4 pts)

Part II - Messing with Classes

5. In the previous homework we used a loop to repeatedly do simulations around the estimate of a sample slope. The setup was:

   - Recall we assume the following model for SLR:

     $$Y_i = \beta_0 + \beta_1 x_i + E_i$$

     where the E_i are assumed to be indendent and identically distributed from a Normal distribution with mean 0 and variance σ^2. Let's assume $\sigma^2 = 1$ for simplicity.

- We can generate data from this model by assuming values for β_0, β_1, and n, along with a sequence of x values.

Here is code to complete that part:

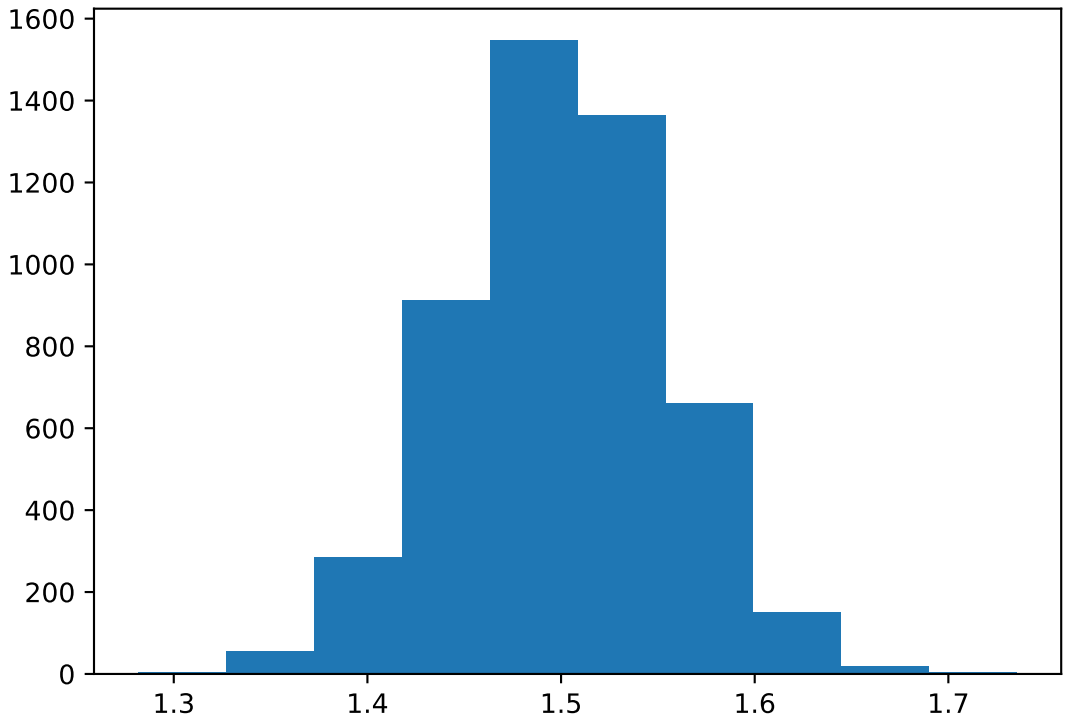
```
#import some modules needed
import matplotlib.pyplot as plt
import numpy as np
from numpy.random import default_rng
from sklearn import linear_model

rng = default_rng(32)
beta_0 = 7
beta_1 = 1.5
# get three 'values' of x at each integer from 0 to 10.
x = np.array(list(np.linspace(start = 0, stop = 10, num = 11))*3)
n = len(x)

#prepare for the LR fit
reg = linear_model.LinearRegression()
#Create a reg object

#initialize array to save estimates (we don't need beta0 but I'm keeping it!)
beta_array = np.zeros(shape = (5000, 2))
for i in range(5000):
    #create the 'responses' modeled from the line plus a random deviation
    y = beta_0 + beta_1*x + rng.standard_normal(n)
    #find the least squares solutions and store them
    fit = reg.fit(x.reshape(-1, 1), y)
    beta_array[i, :] = [fit.intercept_, fit.coef_[0]]

#visualize the distribution of the slopedata
plt.hist(beta_array[:, 1])
plt.show()
```




```
#estimate the probability we are larger than 1.65
bool_prob = beta_array[:, 1] > 1.65
bool_prob.mean()
```

```
## 0.0038
```

Instead of writing this simulation in this way, we'll design a Python class that encapsulates the simulation of the sampling distribution of the slope estimator!

Create a class called `SLR_slope_simulator`:

- Initialize the class using `__init__` with arguments `self`, `beta_0`, `beta_1`, `x`, `sigma`, and `seed`
  - Create initial attributes of `beta_0`, `beta_1`, `sigma`, `x`, `n`, `rng`, and `slopes` (an empty list)
- Required Methods. Your class should include methods (not `@classmethod`, just methods that work on an already created instances of the class):
  - `generate_data` method that generates one dataset (returning x and y in whatever manner you deem fit, i.e. as an array, two arrays, etc.)
  - `fit_slope` method that takes in an `x` and `y` and fits the SLR model, returning the estimated slope
  - `run_simulations` method that takes in a number of simulations argument and uses the `generate_data()` and `fit_slope()` methods within a for loop. This should not return anything but should modify the `slopes` attribute (replaces it with an array of slope estimates)

- `plot_sampling_distribution` method that checks if the `slopes` attribute has length greater than 0 (if it doesn't print a message that `run_simulations()` must be called first). If it is, it should produce a histogram of the slopes approximating the sampling distribution.
- `find_prob` method that takes in a `value` and a `sided` argument. It should checks on the `slopes` attribute as above. If the length is bigger than 0, it should approximate the probability of being "above", "below", or "two-sided" (values for `sided`).
  * If `sided` is "above", you should approximate the probability of being larger than the value.
  * If it is "below", you should approximate the probability of being smaller than the value.
  * If it is "two-sided", you should check if the value is above or below the median. If above, you should find two times the probability of being larger, if below, you should find two times the probability of being smaller.

- Below you definition of your class and its methods, add a section of code that
  - Creates an instance of the object with `beta_0 = 12`, `beta_1 = 2`, `x = np.array(list(np.linspace(start = 0, stop = 10, num = 11))*3)`, `sigma = 1`, and `seed = 10`
  - Call your `plot_sampling_distribution()` method (this should return the error message)
  - Run 10000 simulations
  - Plot the sampling distribution
  - Approximate the two-sided probability of being larger than 2.1
  - Print out the value of the simulated slopes using the attribute

Submission

Now you can grab the 'share' link and submit that! Please make sure to change the sharing settings so that anyone with the link can see the notebook. Good luck and let us know if you run into issues.